The Impact of AI-Generated Solutions on Software Architecture and Productivity: Results from a Survey Study

Giorgio Amasanti and Jasmin Jahić(✉)

University of Cambridge, Cambridge, UK
jj542@cam.ac.uk

Abstract. AI-powered software tools are widely used to assist software engineers. However, there is still a need to understand the productivity benefits of such tools for software engineers. In addition to short-term benefits, there is a question of how adopting AI-generated solutions affects the quality of software over time (e.g., maintainability).

To provide some insight on these questions, we conducted a survey among software practitioners. We conclude that AI tools significantly increase the productivity of software engineers. However, the productivity benefits reduce as projects become more complex. The results also show that there are no significant negative influences of adopting AI-generated solutions on software quality, as long as those solutions are limited to smaller code snippets. However, when solving larger and more complex problems, AI tools generate solutions of a lower quality, indicating the need for architects to perform problem decomposition and solution integration.

Keywords: Software Architecture · AI · Productivity · Quality

1 Introduction

According to a study conducted by Stack Overflow in May 2024 [24], 76.6% developers are using or planning to use AI tools. Engineers use AI tools for various tasks in software engineering [7,28] (e.g., code generation [6], assistance with software architecture [12]). However, adopting AI in software engineering also presents challenges, as AI is a disruptive technology [13]. Therefore, many practitioners (e.g., architects and managers) need to be convinced that adopting AI will bring concrete benefits without hidden "costs". E.g., while some AI tools certainly can provide short-term benefits (e.g., coding assistance), adopting AI-generated solutions might have negative long-term effects (e.g., adopting suboptimal AI-generated code). In software architecture, we label suboptimal solutions as antipatterns and aim to avoid them. Hence, there is a need to understand how the adoption of AI tools (and solutions created with or generated by AI tools)

D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 89–104, 2026.
https://doi.org/10.1007/978-3-032-04403-7_10

influences software architecture in terms of maintainability, extendability, understandability, modifiability, cohesion, coupling, performance, and overall software partitioning. Therefore, we created a study driven by these two main research questions: i) **how AI influences the productivity of software engineers**, and ii) **does adopting AI-generated solutions have significant negative effects on the quality of software architecture**.

To gather data that can help us answer these questions, we conducted a survey among 40 practitioners who use AI tools. The data gathered shows that using AI tools significantly boosts productivity in the early stages of software projects and that adopting AI generated solutions does not affect software architecture in a significantly negative way. However, once there is an established code base, adopting solutions generated by AI (e.g., changing code, new design solutions) is not straightforward and can introduce significant overhead. Despite these challenges, the overwhelming majority of the surveyed professionals reported a significant increase in productivity (over 35%) when using AI tools (for various tasks, not just coding). Therefore, **if you do not use AI tools in software engineering, you are becoming less competitive** because individuals using AI tools ***significantly outperform*** those who do not use such tools for the variety of tasks in Software Development Life Cycle (SDLC).

This paper is organised as follows. In Sect. 2, we discuss related work. In Sect. 3, we present our research methodology. Section 4 presents the results of the industrial survey. We conclude in Sect. 5.

2 Related Work

There exist several studies that investigate how effective AI tools are for code generation [1,6,16]. Engineers also use AI tools for other purposes: program repair [14,18,29], code summarization [2,11], and generating software architecture solutions [12,21]. In addition, there are several studies that investigate quality of solutions that these tools provide [20], indicating that these tools suffer from low accuracy and low reliability [8,9,17,26]. For example, how ChatGPT struggles with lengthy descriptions of problems [25].

However, more recent studies [19,22], such as the one performed by Stack Overflow [24] offer different perspective and indicate that using AI tools in software engineering offers productivity boosts. For example, one of the highest-performing recent AI tools, LDB (a novel AI code debugging framework using GPT-4o), can solve up to 98.2% of the problems in HumanEval benchmark after one attempt (Pass@1) [30]. Bencheikh et al. [5] conducted a study comparing the quality of AI and human-written software requirements, and concluded that the AI "performs reasonably well" as a software requirements engineer. As outlined by He et al. [10], the subject of much research on AI tools for software development has been focussed on the implementation of multi-agent systems. This involves utilising multiple AI entities to work together and perform separate tasks within the software engineering process, much like a human engineering team. This framework is the methodology adopted by many dedicated AI software development tools including ChatDev and MetaGPT, with a seemingly

positive impact on performance in the software development process. Waseem et al. [27] conducted a case study similar to ours investigating the viability of ChatGPT as a software engineering tool. Their study reported that AI was effective at: identifying requirements, streamlining development via code generation, bug detection and architectural decision making. Compared to the existing work, we focus on productivity and quality of software design, when adopting AI-generated solutions.

3 Research Study Design

Our study is composed out of two substudies. In the first sub-study, we created an instance of the classic Tetris videogame with the help of AI tools. We used this sub-study to scan the market for available AI tools and to get initial results of how these tools behave when integrated into Software Development Life Cycle (SDLC). We used the conclusions and experiences from the first substudy to setup a survey with industry professionals (second substudy). To structure our study, we used Goal-Question-Metric (GQM) approach [4,23]. For the design of the survey, we followed the established guidelines [15]. To analyse the data gathered in the survey, we applied the principles of grounded theory using open and axial coding.

3.1 Tetris Project Study: Design

Table 1 presents the detailed design (using the GQM approach) for our substudy in which we created an instance of a classical Tetris game. In this substudy, we aimed to investigate how AI tools behave when integrated in SDLC compared to the work done without using AI tools. To conduct the study, we sought a junior *software engineer* with proficiency in at least one programming language, and an experienced software engineering professional to serve as the *project commissioner*, responsible for defining the business architecture and high-level requirements. The development was organised using the Scrum framework[1].

3.2 Tetris Project Study: Results

The substudy was conducted over a period of six weeks. In the first half of the study, the *software engineer* scanned the market for available AI-tools (Google search, systematic literature reviews, case studies, and reports on using AI in software engineering) and used those tools to experiment with setting up tool chains to cover stages of the Software Development Life Cycle (SDLC). In the second half of the study, over a period of the next three weeks (comprising three one-week sprints), the *software engineer*, guided by instructions from the *project commissioner*, developed three parallel versions of the Tetris game, all three based on the same set of requirements. Each version followed a different technical

[1] https://www.scrum.org/resources/what-scrum-module.

Table 1. GQM Mapping: Tetris Project Study

Goal	Question	Metric
G1: Identify AI tools for SDLC	**Q1.1:** Which AI tools are currently available across SDLC?	**M1.1:** Tool name and SDLC stage(s) supported; context of use
	Q1.2: Which SDLC tools have AI-enabled features?	**M1.2:** Tool name and vendor; list of AI subcomponents/features
	Q1.3: Are there integrated AI-powered SDLC toolchains?	**M1.3:** Toolchain structure; SDLC stages covered
G2: Analyze suitability of AI for SDLC	**Q2.1:** What tasks are commonly assisted or automated by AI?	**M2.1:** Task–tool mapping; automation level
	Q2.2: Which SDLC stages are difficult to delegate to AI?	**M2.2:** Stages flagged as unsuitable; challenges reported per stage
G3: Assess productivity impact of AI tools	**Q3.1:** Does AI reduce time required for specific tasks?	**M3.1:** Estimated time saved per task/project
	Q3.2: How often do AI outputs require rework?	**M3.2:** Percentage of AI outputs needing edits; rework frequency
	Q3.3: How do developers assess productivity with AI?	**M3.3:** Self-rated productivity scores with/without AI
G4: Examine architectural implications of adopting AI-generated solutions	**Q4.1:** How correct and adequate are AI solutions?	**M4.1:** Percentage of AI outputs used without modification; number of bugs
	Q4.2: Are AI-generated solutions harder to modify or test?	**M4.2:** Time to understand/edit/integrate AI code; difficulty rating
	Q4.3: How does AI-generated code affect quality of architecture?	**M4.3:** System consistency; integration issues; maintainability issues; extendibility issues; understandability issues; cohesion and coupling issues; logical partitioning issues

setup. In **version a)**, the *software engineer* did not use any AI tools. In **version b)**, the engineer used GPT Pilot—a terminal-based code planning and generation tool leveraging GPT-4o-mini-2024-07-18 via OpenAI's API—and ChatGPT running on GPT-4o-2024-05-13. In **version c)**, the *software engineer* employed multiple ChatGPT instances (GPT-4o-2024-05-13) to simulate distinct roles: (i) project manager—configures other roles; (ii) requirements engineer—generates refined requirements; (iii) software architect—designs the base software architecture; (iv) developer agent—writes and debugs code; and (v) quality assurance (QA)/quality control (QC) engineer—evaluates and provides feedback on code quality. The *software engineer* manually copied inputs and outputs between these roles to advance development. The results of the study are summirized in Table 2, and the full and raw results, as well as the code, are available online[2345].

[2] https://github.com/AItoolsSE/CAIN-AI-documents.
[3] https://github.com/AItoolsSE/CAIN-AI-approach-A.
[4] https://github.com/AItoolsSE/CAIN-AI-approach-B.
[5] https://github.com/AItoolsSE/CAIN-AI-approach-C.

Table 2. Mapping GQM Structure to Tetris Project Results

G	Q	M	Finding/Observation
G1	Q1.1	M1.1	Identified: ChatGPT, StarCoder, Llama; GitHub Copilot, Cursor IDE; Snyk, Harness; Datadog, Splunk, Logz.io, Moogsoft
	Q1.2	M1.2	Features observed in tools like GPT Pilot, with multi-stage AI support
	Q1.3	M1.3	Tools like GPT Pilot and ChatDev integrate multiple SDLC stages including requirements, coding, and review
G2	Q2.1	M2.1	AI is effective in early implementation and boilerplate generation. Less effective in bug fixing and integration
	Q2.2	M2.2	Bug fixing and code modification stages require multiple prompts; productivity drops as complexity increases
G3	Q3.1	M3.1	AI improves productivity by up to 35%, especially during early-stage development. Long response times from AI tools have a significant negative impact on productivity. As code complexity increases, AI response times also increase, reducing productivity gains
	Q3.2	M3.2	Typically 2–3 prompt iterations are needed; the need for rework increases with code complexity (to resolve introduced bugs)
	Q3.3	M3.3	Asking for AI assistance on small, focused problems maximizes productivity. Attempting to solve large problems in a single prompt often leads to errors. Manual editing is often faster for integration and bug fixes (AI often requires multiple prompts). Humans comprehend the code better than AI
G4	Q4.1	M4.1	AI-generated code is almost always syntactically correct. AI rarely produces functionally adequate results on first attempt and the functional adequacy of generated code decreases as complexity grows
	Q4.2	M4.2	The logical style of AI-generated code—such as naming consistency, modifiability, and line length—is generally comparable to human-written code, contributing to good readability and understandability. AI requires more time than humans for editing and debugging as it often requires multiple prompts. Integrating AI-generated code into an existing codebase can require substantial manual effort, such as renaming variables or adapting function signatures and module interfaces
	Q4.3	M4.3	Large AI-generated code has poor modularity, maintainability, extensibility, and understandability due to illogical structure. Small snippets yield high cohesion, low coupling and a clear separation of concerns, resembling human-written code in architectural quality. Smaller AI-generated code snippets also match human-written code in terms of maintainability, including structural quality and clarity. AI-generated code demonstrates performance characteristics comparable to those of human-written code, with no significant overhead observed

3.3 Survey with Industry Professionals: Design

We used conclusions from the Tetris Project Study (Sect. 3.2) to create a structure of the survey with industry professionals. Table 3 presents the detailed study design using the GQM approach. To conduct the study, we sought industry professionals who are already using some AI tools in their daily work. We aimed for 30 participants, with a variety considering the institution size, years of experience, roles in SDLC, and geography locations. We invited professionals from our network and shared the survey on LinkedIn.

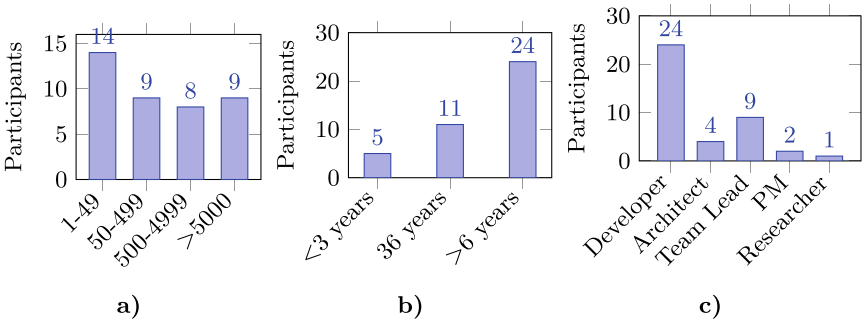

Fig. 1. a) institution size, b) experience (PM - Project Manager), c) role

4 The Impact of AI-Generated Solutions on Software Architecture and Productivity

4.1 Demographics

We conducted the survey in November 2024 and received responses from ***40 participants*** (***the raw survey results are available online***[6]) from 13 different countries and companies of various sizes (Fig. 1).

4.2 Results

As the first step in grounded theory, we performed open-coding analysis on the raw qualitative data, breaking them into discrete parts and labelling them with codes that represent the underlying concepts, actions, and concerns. To build

[6] https://github.com/AItoolsSE/CAIN-AI-documents.

Table 3. GQM: AI Tools in Software Engineering – Productivity and Architecture

Goal	Question	Metric
G1: Identify AI tools in the SDLC	**Q1.1:** Which tasks in the SDLC are supported by AI?	**M1.1:** List of SDLC tasks (e.g., code generation, test generation, CI/CD, bug fixing)
	Q1.2: Which AI tools are used in daily development?	**M1.2:** Tool names selected (e.g., Copilot, ChatGPT)
G2: Assess the productivity benefits of using AI tools	**Q2.1:** How do software engineers perceive productivity improvements with AI?	**M2.1:** Self-reported productivity impact relative to a reference of up to 35% improvement (ordinal scale: much lower to much higher)
	Q2.2: At which SDLC stage is AI usage most beneficial?	**M2.2:** Agreement on usefulness in early implementation (e.g., generating skeleton code)
	Q2.3: What is the most effective way to apply AI tools during development?	**M2.3:** Agreement on effectiveness of decomposition (e.g., prompts on smaller problems yield better results)
	Q2.4: What factors hinder productivity when using AI tools?	**M2.4:** Processing time; increased latency with code size; bug frequency in larger outputs; average number of prompt repetitions; time to fix issues; manual integration effort
G3: Examine architectural challenges of adopting AI-generated code	**Q3.1:** How syntactically correct is generated code?	**M3.1:** Frequency of syntactic correctness (Always, Often, Sometimes, Rarely)
	Q3.2: How functionally adequate is generated code?	**M3.2:** Self-reported success rate; prompt repetition; influence of complexity
	Q3.3: Does large-scale AI-generated code reduce logical organisation and architectural quality?	**M3.3:** Agreement with poor structure in large AI-generated blocks (low maintainability, extendability)
	Q3.4: Does the use of AI increase architectural erosion (e.g., cohesion, coupling)?	**M3.4:** Agreement on architectural erosion; depends on size and structure of AI-generated code
	Q3.5: Is AI-generated code comparable in performance to human-written code?	**M3.5:** Agreement with performance quality (e.g., minimal overhead)
	Q3.6: Does AI-generated code affect maintainability and modifiability?	**M3.6:** Agreement with difficulty updating/extending AI-generated code
	Q3.7: What kinds of tests are generated by AI tools?	**M3.7:** Presence of advanced testing patterns (e.g., mocks, pixel-level testing)
	Q3.8: How reliable are AI-generated tests?	**M3.8:** Initial test failure rate

Table 4. Survey results: Mapping of analysis results to GQM elements

G	Q	M	Finding/Observation
G1	Q1.1	M1.1	AI tools are used across SDLC for code generation, bug fixing, test generation, CI/CD, and documentation support
	Q1.2	M1.2	Tools reported include ChatGPT, Claude, Copilot, and Cursor IDE, with varying preferences based on user experience
G2	Q2.1	M2.1	Productivity gains range from much higher (considering 35% improvement reference) to none, depending on the SDLC task, developer experience, and task complexity. A significant increase in engineering performance productivity
	Q2.2	M2.2	AI is considered most beneficial in early implementation stages (e.g., boilerplate, skeleton code)
	Q2.3	M2.3	Smaller, well-scoped prompts are more effective; prompting skill is a key enabler of productivity
	Q2.4	M2.4	Manual integration effort, low test reliability, prompt iteration overhead, and poor handling of complex tasks. Minimal challenges for smaller code snippets and smaller problems
G3	Q3.1	M3.1	AI-generated code is often syntactically correct but varies in quality depending on prompt clarity and task complexity
	Q3.2	M3.2	First-attempt functional adequacy is inconsistent; requires iterative prompting, especially for complex problems
	Q3.3	M3.3	Larger AI-generated blocks reduce architectural clarity and maintainability; structure quality degrades
	Q3.4	M3.4	Signs of architectural erosion (lower cohesion) in AI-generated code for larger problems and code bases. Minimal negative effect for smaller problems and small code snippets
	Q3.5	M3.5	Performance of AI-generated code is acceptable for simpler cases. It can be affected as the problem size increases
	Q3.6	M3.6	Maintainability can suffer due to lack of structure, contextual gaps, and ambiguous logic
	Q3.7	M3.7	Tests generated are often superficial; lack coverage based on requirements or edge cases
	Q3.8	M3.8	High initial failure rate of tests noted; AI tends to generate tests to pass existing code, not validate correctness

meaningful connections between data fragments in the survey, we applied axial coding on the results of the open-coding analysis. Finally, we mapped these insights to our GQM table (Table 4).

From the results obtained by the application of grounded theory, we conclude the following. The most common uses of AI considering SDLC are to generate small code snippets, modify existing code, fix bugs, and generate tests cases (Fig. 2). The most popular AI tools in the industry are ChatGPT and Copilot (Fig. 3). We observed that 50% of the survey participants who use ChatGPT also use Copilot.

Measuring ***productivity*** in software engineering is not easy [3]. Therefore, we decided to ask for overall ***subjective feeling of productivity*** that each survey participant has when using AI tools. In that subjective context, we did try to understand if AI is more helpful with some tasks and if it introduces overhead in others. Based on our experience from the test project and based on the experiences reported in related work, we decided to differentiate between i) applying AI tools to create a fresh code base and ii) applying AI to an already established project. The results of the survey are presented in a way that reflects this differentiation.

When considering 35% increase in productivity as a reference value (as observed in our test project), the results (Fig. 4) show that 79% of the surveyed participants experience the same, higher or much higher productivity when using AI tools. It is important to note that 21% of the participants experienced lower improvements in productivity or no improvements in productivity at all. We analysed the results of those that indicated lower productivity, but could not find any correlation with other answers. These participants use ChatGPT and Copilot (the same as those who experienced increase in productivity), and use these tools for similar tasks. The only interesting thing here might be that all the juniors (below 3 years of work experience) said that AI tools increase their productivity (none of the juniors was among 21% of the participants who experienced lower improvements in productivity or no improvements in productivity). Majority of the survey participants agree that the highest productivity increase with AI tools can be achieved to generate a skeleton of an application and generate the basic features of the application (Fig. 5).

The overwhelming majority of the survey participants agree that the best way to use AI tools to improve productivity is to break a problem into small chunks and ask AI to implement it (Fig. 6-a). As indicated in Fig. 6-b, 40% survey participants observed increase in processing time of AI tools as their code size increases, affecting productivity in a negative way. Most of the survey participants (more than 67%) observed that AI tools are more likely to break the existing code, hindering the productivity, as the size of the code base increases (Fig. 6-c).

Although 65% of the survey participants agree that AI can take more time than humans to change the code (Fig. 7-a), there is a strong indication from 15% of the participants who disagree with this claim. This is an interesting result, as it shows that AI in certain scenarios can be faster than humans when it is necessary to change a code base. When it comes to fixing bugs, the results of the survey (Fig. 7-b) show that 20% of the participants experienced that AI is faster than humans. However, 60% of the participants still experience that humans are

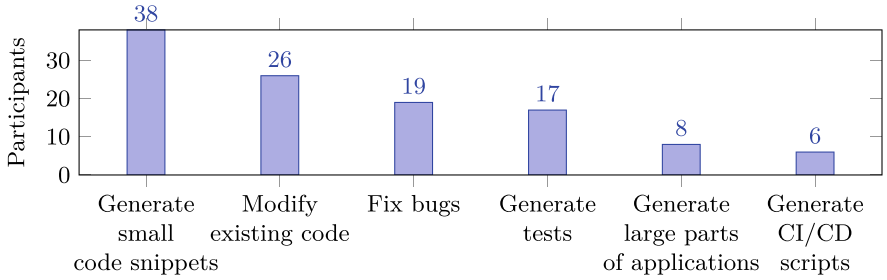

Fig. 2. Usage of AI for Coding Tasks

(much) faster when it comes to fixing bugs. The survey results (Fig. 7-c) show that there is a notable number of people who expressed that they required a significant effort when integrating AI-generated code into existing code. However, there are also those that consider this not to be a challenge at all (over 35% of the surveyed participants).

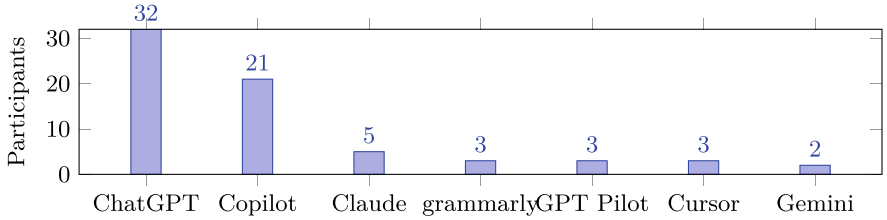

Fig. 3. AI tools used by the survey participants (there were also single users of Amazon Q, CodeWhisperer, Open-source projects, in-house tools)

Some survey participants claim challenges with the code organization of the generated code (Fig. 8-a). Although, it seems that there were few (15%) who had positive experiences with AI partitioning the code (from the answers we had in the survey, we could not find any correlation between these results and any other answers - except that all those who answered in this way have over 3 years of experience in software engineering). The survey participants agree that generating small snippets of code with AI tool and their adoption does not lead to architecture erosion in terms of cohesion, coupling, and logical code partitioning (Fig. 8-b). It seems that over 50% of the survey participants agree that AI generated code is performant (no significant unnecessary overheads in the code that humans would avoid) in a similar way that it would have been if it was written by humans (Fig. 8-c). It is important to emphasize here that 17.5% of the survey participants have different experience and disagree with this claim, indicating that AI tools still have some work to do when it comes to code performance. However, from the answers we gathered, we could not find any correlation with other parameters that would indicate why this is the case

(similar tools, years of experience, company size, and roles as for those that have experienced the opposite).

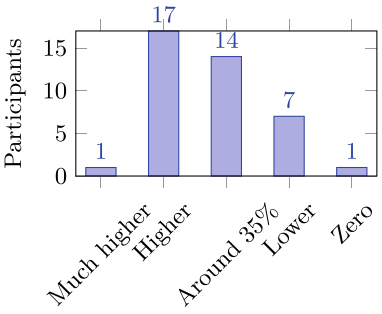

Fig. 4. Perceived productivity improvements (reference: 35%)

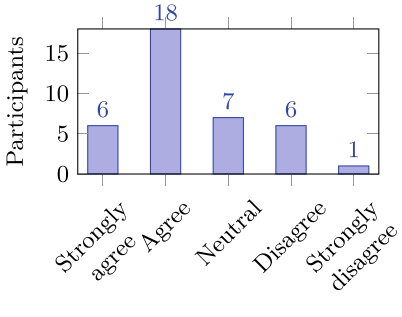

Fig. 5. The best moment to apply AI is in the early implementation

It seems to be a common experience in industry that first shot answers provided by AI tools rarely fulfill desired functionality, and therefore prompts have to be tuned (Fig. 9-a). Over 50% of the survey participants say that AI-generated code is almost always syntactically correct. But, there is a significant number of participants who disagree (Fig. 9-b). However, from the answers we gathered, we could not find any correlation with other parameters that would indicate why this is the case (similar tools, years of experience, company size, and roles as for those that have experienced the opposite). When it comes to the tests, some survey participants agree that AI generated tests use more complex testing techniques (around 20%), while the same number of the survey participants disagree with this claim. The majority was neutral with respect to this question (Fig. 9-e).

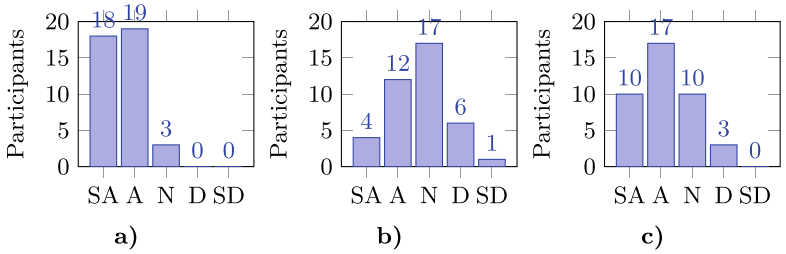

Fig. 6. Participant agreement levels: (a) The best way to use AI tools for improving productivity is to break a problem into small chunks and ask AI to implement it; (b) As code size and code complexity increase, productivity gains decrease because AI tools take a longer time to process and return the larger code segments; (c) As code size and code complexity increase, productivity gains decrease because AI tools are more likely to break existing code. Response scale: SA = Strongly Agree, A = Agree, N = Neutral, D = Disagree, SD = Strongly Disagree.

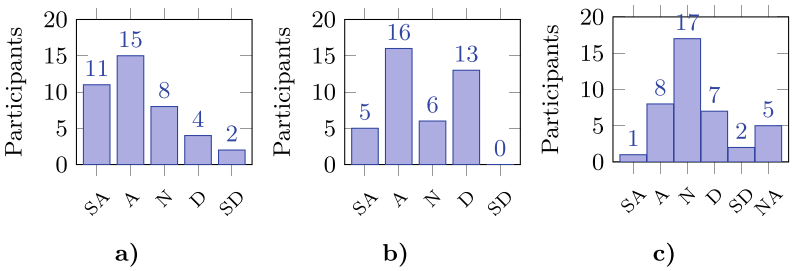

Fig. 7. Participant agreement levels on AI limitations for code understanding and integration: (a) Using AI to make code changes can be more time-consuming than doing it manually; (b) AI can take significantly longer to fix bugs than humans; (c) AI-generated code requires significant integration effort. Response scale: SA = Strongly Agree, A = Agree, N = Neutral, D = Disagree, SD = Strongly Disagree.

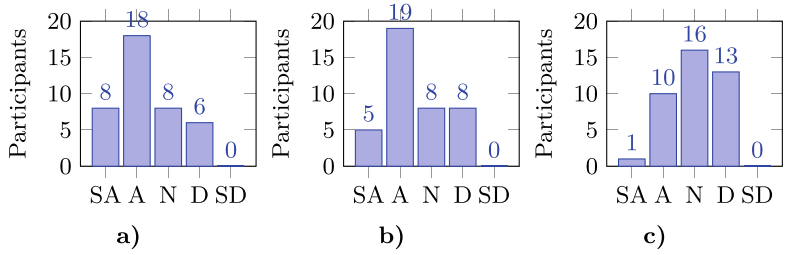

Fig. 8. Perceived quality of AI-generated code: (a) When generating full applications or large portions of code involving multiple functions/classes at once, partition and organisation of AI-generated code can be of a significantly lesser quality to that of human-written code and therefore not easy to maintain; (b) Code snippets (smaller code chunks) generated by AI are of comparable maintenance quality (high cohesion, low coupling) to that of human-written code; (c) Code generated by AI is of comparable performance quality to that of human-written code. Response scale: SA = Strongly Agree, A = Agree, N = Neutral, D = Disagree, SD = Strongly Disagree, NA = Not Applicable.

When it comes to **individual comments** we received about **positive effects of AI tools on productivity**, the survey participants listed several interesting statements (we tried to merge similar ones). For example: i) use LLMs to quickly sum up and recommend the latest cutting-edge research that's relevant to the daily work, which really speeds up getting new tech into software. They claim that this has been more useful than help with coding; ii) Quickly access information from design documents and other company-specific resources (much faster than standard searching); iii) Brainstorm ideas, ask questions like "how do I do this?" rather than asking the tool to output code; iv) Ask AI tools for example implementation with some libraries is faster than reading documentation; v) Generate boilerplate code.

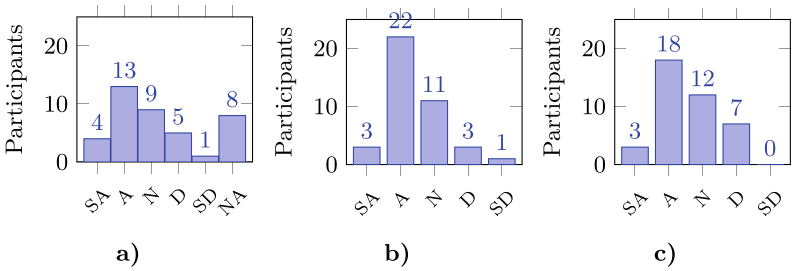

Fig. 9. Participant responses on characteristics of AI-generated code: (a) AI-generated code rarely fulfils desired functionality in the first shot. As the problem size increases, so does the need for fixing the generated code, (b) AI-generated code is almost always syntactically correct, (c) Tests generated by AI tend to use more complex testing techniques (e.g., mock objects, pixel colour testing). Response scale: SA = Strongly Agree, A = Agree, N = Neutral, D = Disagree, SD = Strongly Disagree, NA = Not Applicable.

4.3 Summary

Here are the conclusions based on the survey analysis:

i) About 45% of the survey participants experienced an increase in productivity of OVER 35% when using AI tools. This fact cannot be ignored, and it is clear that anyone working today in the software industry needs to use AI tools if they want to remain competitive. Otherwise, with the best of their efforts, they will struggle to keep up with the productivity that those who use AI tools can achieve.

ii) Adoption of AI-generated code snippets does NOT lead to a significant erosion of software architecture (in terms of cohesion, coupling, partitioning of logical parts). The code is performant and easy to maintain. AI-generated code is almost always syntactically correct and the quality of the logical syntax style is adequate.

iii) AI tools can suggest adequate architectural designs. However, this conclusion should be taken carefully, as it depends on many factors, including how we formulate the problem and the experience of those that consume the AI generated solutions, as solutions often require tweaks. Junior software architects would certainly struggle to assess whether AI-generated solutions are adequate.

iv) However, AI tends to suggest inadequate solutions when faced directly with large and complex problems (without prior decomposition). In addition, AI tends to suggest inadequate solutions when changing complex and large code bases. When applied on large and complex problems, AI-generated code is often of a significantly lesser quality than human-written code in terms of partition and organisation and, therefore, not easy to maintain and integrate. Furthermore, using AI to change complex and large code bases leads to the breaking of existing code.

Although engineers tend to profit from AI in terms of higher productivity, it is also clear that they still need to perform a decomposition of problems to the level that AI can handle. Furthermore, it is also clear that the job of software engineers is to synthesise the generated solutions into well-architected systems. Therefore, it is fair to say that architecture skills are highly needed in the age of AI.

5 Conclusion and Future Work

Generating code and tests is only one of the use cases where the adoption of AI tools increases productivity. Others include using AI for more conceptual work (software architecture) and using AI to help with side tasks (e.g., summarising information). Despite all the challenges associated with the use of AI in software engineering, it is clear that AI is not a hype and that AI tools in software engineering are here to stay. The increase in productivity of over 35% is a clear and strong sign that individuals using AI tools significantly outperform those who do not use AI tools. The questions that now open are related to the use cases where we could apply AI, how to reduce friction when adopting AI tools in existing workloads and code bases, and what is it that we can now achieve with the increase in productivity. It is also clear that not everyone equally enjoys increases in productivity, and there is a strong concern that junior engineers might bring more overhead than benefit when adopting solutions that they fully do not understand.

As code size and code complexity increase (in general, as a problem complexity increases), productivity gains decrease because AI tools are a) more likely to break existing code; b) demonstrate increased processing time; c) software shows the sign of architectural erosion. Therefore, it is more challenging to change code using AI than to create new code. We have to observe that today this is where most of the engineering time is invested, and it would be great if AI could support these activities in a smoother way. That will also be one of the topics of our future work.

References

1. OpenAI, Pilipiszyn, A.: https://openai.com/blog/gpt-3-apps/. Accessed 8 Jan 2023
2. Ahmed, T., Devanbu, P.: Few-shot training LLMs for project-specific code-summarization. In: Automated Software Engineering, 2022, NY, USA (2023)
3. de Barros Sampaio, S.C., Barros, E.A., de Aquino, G.S., e Silva, M.J.C., de Lemos Meira, S.R.: A review of productivity factors and strategies on software development. In: 2010 Fifth International Conference on Software Engineering Advances, pp. 196–204 (2010). https://doi.org/10.1109/ICSEA.2010.37
4. Basili, V.R., Weiss, D.M.: A methodology for collecting valid software engineering data. IEEE Trans. Softw. Eng. **SE-10**(6), 728–738 (1984). https://doi.org/10.1109/TSE.1984.5010301

5. Bencheikh, L., Höglund, N.: Exploring the efficacy of ChatGPT in generating requirements: an experimental study (2023)
6. Bucaioni, A., Ekedahl, H., Helander, V., Nguyen, P.T.: Programming with ChatGPT: how far can we go? Mach. Learn. Appl. **15**, 100526 (2024). https://doi.org/10.1016/j.mlwa.2024.100526. https://www.sciencedirect.com/science/article/pii/S2666827024000021
7. Chang, Y., et al.: A survey on evaluation of large language models. ACM Trans. Intell. Syst. Technol. **15**(3) (2024). https://doi.org/10.1145/3641289
8. Ciniselli, M., Cooper, N., Pascarella, L., Poshyvanyk, D., Penta, M.D., Bavota, G.: An empirical study on the usage of BERT models for code completion. CoRR abs/2103.07115 (2021). https://arxiv.org/abs/2103.07115
9. Fan, Z., Gao, X., Mirchev, M., Roychoudhury, A., Tan, S.H.: Automated repair of programs from large language models (2023)
10. He, J., Treude, C., Lo, D.: LLM-based multi-agent systems for software engineering: vision and the road ahead (2024). https://arxiv.org/abs/2404.04834
11. Hu, X., Li, G., Xia, X., Lo, D., Jin, Z.: Deep code comment generation with hybrid lexical and syntactical information. Emp. Softw. Engg. **25**(3), 2179–2217 (2020)
12. Jahic, J., Sami, A.: State of practice: LLMs in software engineering and software architecture. In: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), pp. 311–318. IEEE Computer Society, Los Alamitos, CA, USA, June 2024. https://doi.org/10.1109/ICSA-C63560.2024.00059. https://doi.ieeecomputersociety.org/10.1109/ICSA-C63560.2024.00059
13. Jahić, J., Roitsch, R., Grzymkowski, L.: Knowledge-based adequacy assessment approach to support AI adoption. In: 2021 IEEE 18th International Conference on Software Architecture Companion (ICSA-C), pp. 8–14 (2021). https://doi.org/10.1109/ICSA-C52384.2021.00008
14. Jain, N., et al.: Jigsaw: large language models meet program synthesis. In: Proceedings of the 44th International Conference on Software Engineering, ICSE 2022, New York, NY, USA, pp. 1219–1231 (2022)
15. Kasunic, M.: Designing an effective survey. Technical report (2005)
16. Kim, S., Zhao, J., Tian, Y., Chandra, S.: Code prediction by feeding trees to transformers. In: ICSE, pp. 150–162 (2021)
17. Mastropaolo, A., et al.: Studying the usage of text-to-text transfer transformer to support code-related tasks. CoRR abs/2102.02017 (2021). https://arxiv.org/abs/2102.02017
18. Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M.C., Jiang, Z.M.J.: GitHub Copilot AI pair programmer: asset or liability? J. Syst. Softw. **203**, 111734 (2023)
19. Nguyen-Duc, A., Abrahamsson, P., Khomh, F. (eds.): Generative AI for Effective Software Development. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-55642-5
20. Pearce, H.A., Tan, B., Ahmad, B., Karri, R., Dolan-Gavitt, B.: Can OpenAI codex and other large language models help us fix security bugs? ArXiv abs/2112.02125 (2021)
21. Ramachandran, R.: Transforming software architecture design with intelligent assistants-a comparative analysis. In: SoutheastCon 2025, pp. 1446–1454 (2025). https://doi.org/10.1109/SoutheastCon56624.2025.10971683
22. Sauvola, J., Tarkoma, S., Klemettinen, M., Riekki, J., Doermann, D.: Future of software development with generative AI. Autom. Softw. Eng. **31**(1), 26 (2024). https://doi.org/10.1007/s10515-024-00426-z

23. van Solingen, R., Basili, V., Caldiera, G., Rombach, H.D.: Goal Question Metric (GQM) approach. John Wiley and Sons, Ltd. (2002). https://doi.org/10.1002/0471028959.sof142. https://onlinelibrary.wiley.com/doi/abs/10.1002/0471028959.sof142
24. StackOverflow: 2024 developer survey insights for AI/ML (2024). https://stackoverflow.co/labs/2024-developer-survey-insights-for-ai-ml/. Accessed 08 Sept 2024
25. Tian, H., et al.: Is ChatGPT the ultimate programming assistant – how far is it? (2023). https://arxiv.org/abs/2304.11938
26. Tufano, M., Watson, C., Bavota, G., Penta, M.D., White, M., Poshyvanyk, D.: An empirical study on learning bug-fixing patches in the wild via neural machine translation. ACM Trans. Softw. Eng. Methodol. **28**(4) (2019)
27. Waseem, M., Das, T., Ahmad, A., Liang, P., Fehmideh, M., Mikkonen, T.: ChatGPT as a software development bot: a project-based study (2024). https://arxiv.org/abs/2310.13648
28. Yang, J., et al.: Harnessing the power of LLMs in practice: a survey on ChatGPT and beyond. ACM Trans. Knowl. Discov. Data **18**(6) (2024). https://doi.org/10.1145/3649506
29. Zhang, J., et al.: Repairing bugs in Python assignments using large language models (2022)
30. Zhong, L., Wang, Z., Shang, J.: Debug like a human: a large language model debugger via verifying runtime execution step-by-step (2024). https://arxiv.org/abs/2402.16906